LLM Environment Open Data: an Open Dataset, API, and MCP Server for the Quantitative Literature on the Environmental Footprint of Large Language Models

Working Paper

March 12, 2026

Abstract

The literature on the environmental footprint of large language models now provides a growing number of quantified indicators, but these results remain scattered across journal articles, model cards, technical reports, and institutional syntheses. This dispersion slows comparison across studies and limits cumulative reuse. We present *LLM Environment Open Data*, a structured dataset that extracts, normalizes, and documents quantitative values reported on LLM training, inference, infrastructure, and lifecycle impacts. The project comprises four complementary artifacts: an open tabular dataset, a layer of comparability metadata, a local HTTP API, and an MCP server for agent-oriented querying. The initial release contains 49 records spanning 11 sources, and links each value to its unit, system boundary, analytical scope, and precise source locator. We further show how this evidence base can support a first environmental externalities estimator at the prompt, software feature, and annualized usage levels. This paper describes the dataset construction process, its structure, its limitations, and its value for both reproducible quantitative reviews of AI environmental impacts and environmentally responsible software design.

Keywords: LLMs, open data, dataset, environmental footprint, API, MCP, quantitative review.

1 Introduction

Debate on the environmental footprint of LLMs has intensified as publications have multiplied on training, inference, water use, data centers, and hardware lifecycle effects [16, 14, 10, 13, 12, 6]. Yet a simple methodological problem remains: quantitative values are distributed across heterogeneous formats, with highly variable units of analysis, system boundaries, and documentary locators. This heterogeneity makes cumulative reuse difficult.

The aim of this paper is to document the data infrastructure associated with our quantitative review of LLMs and the environment. The primary contribution is an open dataset that stores extracted values together with comparability metadata and machine-access interfaces through an API and an MCP server. This infrastructure also serves as the basis for a first externalities estimator for practical use cases, such as estimating the environmental impact of a prompt, a software feature, or an annualized service relying on an LLM.

2 Positioning

The existing literature already provides robust anchor points, for example the training emissions of BLOOM [10], the GPU-hours and emissions documented by Meta for Llama 3.1 [11], production-scale inference measurements at Google [6], water-footprint estimates [9], and annual electricity

trajectories for data centers [7, 15, 8, 4]. However, the literature provides few open infrastructures designed to consolidate such results into a directly reusable form.

The *LLM Environment Open Data* project addresses this gap through three simple design choices:

- one record per quantified value;
- explicit traceability to the page, table, section, or paragraph of origin;
- exposure of the data for both human readers and software agents.

3 Dataset Construction Method

The dataset is built from the quantitative literature review associated with the main publication on LLMs and the environment. For each retained source, we create one record for each quantified value that can be directly reused in the review. A single study may therefore generate several records when it reports multiple metrics or multiple units of analysis.

Each record contains the following fields:

- unique record identifier;
- source key;
- short citation;
- publication year;
- analytical phase: `training`, `inference`, `infrastructure`, `lifecycle`;
- impact category: carbon, energy, water, or compute;
- metric name;
- value and unit;
- model or scope concerned;
- geography;
- temporal scope;
- system boundary;
- data type: measured, calculated, modeled, projected, estimated, or statistical;
- source locator;
- source URL;
- free-text note.

This structure makes it possible to separate three dimensions that are often conflated in narrative reviews: the observed phenomenon, the way it was measured, and the exact level to which it applies.

4 Initial Release Content

The initial release contains 49 records spanning 11 sources. All four analytical phases are represented: 9 records for training, 17 for inference, 17 for infrastructure, and 6 for lifecycle effects. Energy and carbon are the most represented impact categories, followed by water.

Source	Phase	Example metric	Example value
Strubell et al. (2019) [16]	training	training emissions	626 155 lb CO_2e
Luccioni et al. (2023) [10]	training	BLOOM 176B emissions	24.7–50.5 tCO_2e
Meta (2024) [11]	training	Llama 3.1 405B GPU-hours	30.84 million GPU-hours
Elsworth et al. (2025) [6]	inference	energy per prompt	0.24 Wh/prompt
Ren et al. (2024) [13]	inference	energy per 500-word page	0.0195 kWh/page
Li et al. (2025) [9]	lifecycle / water	GPT-3 total training water use	5.4 million liters
IEA (2025) [7]	infrastructure	annual global data center electricity	415 TWh in 2024
Berkeley Lab (2025) [8]	infrastructure	annual US data center electricity	176 TWh in 2023
de Vries-Gao (2025) [4]	infrastructure	annualized global AI power demand	46.4–201.5 TWh/year derived
Morrison et al. (2025) [12]	lifecycle	total water over model creation pipeline	2.769 million liters

Table 1: Representative records from the initial dataset release.

5 Formats and Schema

The dataset is distributed in two synchronized forms: a canonical CSV file and a JSON export. The CSV facilitates human auditing, spreadsheet-based review, and version control. The JSON format facilitates programmatic integration into scripts, agents, or web interfaces. A JSON schema documents the expected structure and defines required fields.

The decision to maintain a canonical tabular source responds to a practical constraint: bibliographic extractions are often first reviewed and corrected manually. JSON is then regenerated from the CSV. This separation avoids multiplying sources of truth.

The main dataset is complemented by a second metadata table keyed by `record_id`. This layer makes explicit estimation-oriented fields such as `unit_of_analysis`, `source_type`, `uncertainty_level`, `raw_or_derived`, and `applicability_domain`. The methodological point is important: these fields are no longer only reconstructed in code, but published as data and therefore remain auditable, editable, and versionable.

6 Local HTTP API

The project provides a local HTTP API for rapid exploration of the dataset. The main endpoints are:

- `/health` to verify service status;
- `/records` to list records with simple filters;
- `/records/<record_id>` to retrieve a specific record;
- `/sources` to list distinct source studies;
- `/stats` to retrieve summary statistics;
- `/estimate` to produce prompt- or request-level estimates;
- `/estimate_feature` to produce annualized software-feature-level estimates.

This API is not intended to replace a full open-data platform. Rather, it provides a minimal programmatic reuse layer directly associated with the manuscript and the data repository [2]. The estimation endpoints extend the role of the dataset: it is no longer only an archive of reported values, but also a support for traceable calculations grounded in literature-derived factors.

7 MCP Server

The project also provides an MCP server over `stdio`. Its purpose differs from that of the API: it enables software agents to query the dataset directly as structured context, without scraping or heuristic reading of tables in the manuscript.

The current version exposes six tools:

- `list_records`;
- `get_record`;
- `aggregate_by_phase`;
- `list_sources`;
- `estimate_externalities`;

- `estimate_feature_externalities`.

This design addresses a precise need: enabling an assistant or software tool to recover a source-backed quantitative value, verify its analytical phase, and trace it back to the underlying publication without returning to an unrestricted reading of the PDF [3].

8 From Data to Estimation

One of the central use cases of the project is the following question: "Here is my prompt with a given LLM. Can you estimate the associated environmental externalities?" To make that question computationally tractable, the dataset must be complemented by comparability metadata and by a factor-selection logic.

The current method follows four simple steps. First, the use case is classified into a relevant family, such as *prompt-level inference* or *query-level inference*. Second, the most appropriate factors are selected from the literature, prioritizing production measurements over report-level estimates. Third, these factors are adjusted to context when the user provides an electricity-mix carbon intensity or a water intensity. Fourth, the results are aggregated at the software-feature level and then over monthly or annual horizons.

The project therefore implements an explicit distinction between observed and derived values. For example, 0.24 Wh/prompt reported by Google [6] remains an observed reference value. By contrast, converting that value into a carbon estimate contextualized to a French electricity mix is a derived value and should be labeled accordingly.

8.1 Final Calculation Equations

Let T_{in} and T_{out} denote the input and output token counts for one LLM interaction, and let $T_{ref} = 1550$ be the reference prompt size currently used by the implementation. The first adjustment is a token-scaling ratio:

$$r_{tok} = \text{clamp}\left(\frac{T_{in} + T_{out}}{T_{ref}}, 0.25, 4.0\right). \tag{1}$$

If E_{prompt} is the literature-based energy factor in Wh per prompt and n_{req} is the number of LLM requests considered in the scenario, the request-level energy estimate is:

$$E_{req}^{central} = E_{prompt} \times r_{tok} \times n_{req}. \tag{2}$$

In the current implementation, the lower bound is set equal to this central value, while the upper bound uses a more conservative query-level factor E_{query}^{high} from the literature:

$$E_{req}^{low} = E_{req}^{central}, \tag{3}$$

$$E_{req}^{high} = E_{query}^{high} \times r_{tok} \times n_{req}. \tag{4}$$

Carbon and water are then computed either from direct literature factors or from contextualized energy conversion. Let CI be the electricity carbon intensity in gCO_2e/kWh and WI the water intensity in L/kWh. The derived conversions are:

$$C(E) = \frac{E}{1000} \times CI, \tag{5}$$

$$W(E) = \frac{E}{1000} \times WI \times 1000. \tag{6}$$

The factor 1000 in the last equation converts liters into milliliters. In practice, the estimator preserves the published prompt-level carbon and water factors as observed anchors, then expands the range with the contextualized values:

$$C_{req}^{central} = C\left(E_{req}^{central}\right), \tag{7}$$

$$C_{req}^{low} = \min\left(C_{lit},\, C\left(E_{req}^{central}\right)\right), \tag{8}$$

$$C_{req}^{high} = \max\left(C_{lit},\, C\left(E_{req}^{high}\right)\right), \tag{9}$$

$$W_{req}^{central} = \max\left(W_{lit},\, W\left(E_{req}^{central}\right)\right), \tag{10}$$

$$W_{req}^{low} = \min\left(W_{lit},\, W\left(E_{req}^{central}\right)\right), \tag{11}$$

$$W_{req}^{high} = \max\left(W_{lit},\, W\left(E_{req}^{high}\right)\right), \tag{12}$$

where C_{lit} and W_{lit} are the direct prompt-level carbon and water factors extracted from the literature.

At the software-feature level, let n_f be the number of LLM requests per feature use, u_m the monthly number of feature uses, and m_y the number of months per year. Annualized feature uses and annualized LLM requests are:

$$U_{year} = u_m \times m_y, \tag{13}$$

$$N_{LLM,year} = U_{year} \times n_f. \tag{14}$$

The annual direct LLM impact is obtained by scaling the per-request range:

$$E_{LLM,year} = E_{req} \times N_{LLM,year}, \tag{15}$$

$$C_{LLM,year} = C_{req} \times N_{LLM,year}, \tag{16}$$

$$W_{LLM,year} = W_{req} \times N_{LLM,year}. \tag{17}$$

Finally, let s_i denote the software overhead in Wh per feature use for technical component i. The annual overhead of each component is:

$$E_{i,year}^{soft} = s_i \times U_{year}, \tag{18}$$

$$C_{i,year}^{soft} = C\left(E_{i,year}^{soft}\right), \tag{19}$$

$$W_{i,year}^{soft} = W\left(E_{i,year}^{soft}\right), \tag{20}$$

and the total annual software overhead is the sum over all components. The final annual result returned by the tool is therefore:

$$E_{total,year} = E_{LLM,year} + \sum_i E_{i,year}^{soft}, \tag{21}$$

$$C_{total,year} = C_{LLM,year} + \sum_i C_{i,year}^{soft}, \tag{22}$$

$$W_{total,year} = W_{LLM,year} + \sum_i W_{i,year}^{soft}. \tag{23}$$

These equations formalize the current implementation of the API and MCP estimator and clarify the distinction between source-backed factors and derived contextualized values.

9 Research Questions

The transition from evidence base to operational estimation leads to four research questions:

- Which metadata are required to make published environmental factors genuinely comparable and reusable?
- To what extent can LLM externalities be estimated from the literature when direct provider telemetry is unavailable?
- How should LLM calls be integrated into a broader software environmental accounting boundary?
- How does machine-access exposure through an API and MCP improve practical reuse for environmentally responsible AI and software design?

10 Implementation Example: From Prompt to Software Feature

The current release includes a first-level estimator designed for screening use. For a standard text interaction, it first combines the most reusable inference factors in the corpus, notably 0.24 Wh/prompt, 0.03 gCO_2e/prompt, and 0.26 mL/prompt from production-scale measurements reported by Elsworth et al. [6], together with the 2.9 Wh/query order of magnitude reported by EPRI [5] in order to construct a more conservative upper bound.

From these factors, the API and the MCP return a structured result including low, central, and high estimates, the selected factors, and the applied assumptions. For a demonstration scenario corresponding to a text interaction with 1 200 input tokens and 350 output tokens under a simplified French electricity mix, the estimator returns 0.24 to 2.9 Wh per request, 0.0096 to 0.116 gCO_2e, and 0.096 to 1.16 mL of water.

The same engine can then be used at the software-feature level. For a conversational service used 10 000 times per month over twelve months, with one LLM call per use and an additional software overhead of 0.15 Wh per use, the annualized estimate reaches 46 800 to 366 000 Wh, 1 872 to 14 640 gCO_2e, and 18 720 to 146 400 mL of water. The important point is not only the final figure, but the decomposition between direct LLM impact and software-system overhead.

This implementation is deliberately conservative. It does not claim to provide an auditable environmental assessment in the sense of formal carbon accounting. Rather, it provides a traceable, reproducible, and extensible estimate that is useful for eco-design, use-case prioritization, and the later development of more robust methods for software environmental accounting.

11 Evaluation Across Three Software Scenarios

To assess the usefulness of the proposed framework, we applied the estimator to three representative software scenarios: a support chatbot, a retrieval-augmented assistant, and a batch-generation workflow. All scenarios rely on the same estimation logic, but differ in token volume, number of LLM calls per use, monthly usage volume, and software overhead.

Scenario	Annual volume	Total annual energy	Total annual carbon	Total annual water
Support chatbot	120 000 uses/year	46 800–366 000 Wh	1 872–14 640 gCO_2e	18 720–146 400 mL
RAG assistant	48 000 uses/year	61 737.6–506 553.6 Wh	2 467.2–20 265.6 gCO_2e	24 691.2–202 617.6 mL
Batch generation	600 000 uses/year	410 340–4 426 080 Wh	16 440–177 060 gCO_2e	164 160–1 770 420 mL

Table 2: Demonstration results across three software scenarios.

These results illustrate three properties of the method. First, moving from unit-level usage to annualized software features substantially changes the relevant order of magnitude for decision-making. Second, non-LLM software overhead is not negligible and should be tracked explicitly. Third, the spread between low and high bounds remains large, confirming that the tool should be used as a screening and relative-comparison instrument rather than as a definitive measurement device.

12 Scientific Contribution

The main contribution is not algorithmic but infrastructural. The publication associated with the dataset transforms a literature review into a reusable evidence base. This changes the nature of the scientific deliverable in four ways.

First, it separates the narrative layer of the paper from the reusable factual layer. Second, it enables incremental updates of the extraction base without rewriting the entire synthesis. Third, it provides a common support for comparing results that belong to different analytical levels, such as a query, a prompt, a generated page, a final training run, or an annual data center electricity figure. Fourth, it serves as an operational basis for an externalities estimator for software systems integrating LLMs, thereby bringing the quantitative review closer to the concerns of responsible digital design and software environmental accounting.

13 Limitations

The dataset does not eliminate the underlying heterogeneity of the literature; it makes that heterogeneity explicit. Values remain dependent on assumptions, calculation methods, and system boundaries. Some sources are peer-reviewed papers, while others are model cards or technical reports. The dataset therefore includes metadata on evidence type and scope precisely to reduce invalid comparisons.

The current release is also partial. It primarily covers the values mobilized in the associated review paper and the major sources identified so far. The objective is not to claim immediate exhaustiveness, but to provide an open infrastructure that can be enriched, audited, and corrected over time.

14 Conclusion

This paper accompanies the release of *LLM Environment Open Data*, a structured dataset on quantified environmental impacts of LLMs and their infrastructure. By combining an open dataset, a layer of comparability metadata, an HTTP API, and an MCP server, it proposes an augmented form of scientific publication: readers receive not only a synthesis, but also the evidence layer that makes that synthesis verifiable, extensible, and automatable [1, 2, 3].

The next logical step is to expand the source corpus and publish the dataset with a persistent identifier. At this stage, the project already provides the necessary scaffolding to treat the dataset as a scientific object in its own right, while also serving as the starting point for a method of environmental accounting for software systems integrating LLMs.

References

[1] Apachot and Codex. Llm environment open data dataset, 2026. Local dataset project in repository ‘llm-environment-opendata‘; replace with GitHub or DOI URL at release time.

[2] Apachot and Codex. Llm environment open data http api, 2026. Local software artifact in repository ‘llm-environment-opendata‘; replace with GitHub release URL at release time.

[3] Apachot and Codex. Llm environment open data mcp server, 2026. Local software artifact in repository ‘llm-environment-opendata‘; replace with GitHub release URL at release time.

[4] Alex de Vries-Gao. Artificial intelligence: Supply chain constraints and energy implications. *Joule*, 9(6):1153–1156, 2025.

[5] Electric Power Research Institute. Power demand for data centers and artificial intelligence, 2024.

[6] Catherine Elsworth, Kuan Huang, David Patterson, Ian Schneider, et al. Measuring the environmental impact of delivering ai at google scale. *arXiv preprint arXiv:2508.15734*, 2025.

[7] International Energy Agency. Energy and ai, 2025.

[8] Lawrence Berkeley National Laboratory. Report evaluates increase in electricity demand from data centers, 2025.

[9] Pengfei Li, Jianyi Yang, Md Arafat Islam, and Shaolei Ren. Making ai less “thirsty”: Uncovering and addressing the secret water footprint of ai models. *Communications of the ACM*, 68(4):46–54, 2025.

[10] Alexandra Sasha Luccioni, Sebastien Viguier, and Anne-Laure Ligozat. Estimating the carbon footprint of BLOOM, a 176b parameter language model. *Journal of Machine Learning Research*, 24(253):1–15, 2023.

[11] Meta. Llama 3.1 model card, 2024.

[12] Jack Morrison, Christine Na, Jose Fernandez, Tim Dettmers, et al. Holistically evaluating the environmental impact of creating language models. *arXiv preprint arXiv:2503.05804*, 2025.

[13] Siyuan Ren, Bill Tomlinson, R. W. Black, and Andrew W. Torrance. Reconciling the contrasting narratives on the environmental impact of large language models. *Scientific Reports*, 14:28180, 2024.

[14] Matthias C. Rillig, Martina Ågerstrand, Min Bi, Kerry A. Gould, and Uli Sauerland. Risks and benefits of large language models for the environment. *Environmental Science & Technology*, 57(9):3464–3466, 2023.

[15] Arman Shehabi, Andrew Newkirk, Sarah J. Smith, Alex Hubbard, Nhat Lei, et al. 2024 united states data center energy usage report. Technical report, Lawrence Berkeley National Laboratory, 2024.

[16] Emma Strubell, Ananya Ganesh, and Andrew McCallum. Energy and policy considerations for deep learning in nlp. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 3645–3650, 2019.